

Using an LLM as a Judge for Tool-Calling Validation

We can use a **second LLM as a Judge/Validator** to validate the tool selected by the first LLM.

The basic idea is:

LLM 1 selects the tool → LLM 2 validates whether the selected tool is correct → Execute the tool only if validation passes.

Example

LLM 1 - Tool Selection

User Query:

You are an Employee Assistant. Select the appropriate tool for the user's request.
Can you give me my leave balance?

Available Tools:

- GetEmployeeLeaveBalance
- GetSalary
- GetTaxDetails
- etc.

LLM 1 Response:

Selected Tool: GetEmployeeLeaveBalance

LLM 2 - Tool Validator / Judge

Now we send the original query, selected tool, and available tools to another LLM.

System Prompt:

You are a Tool Validator. Validate whether the tool selected by LLM 1 is appropriate for the user's query.

Input:

User Query:
Can you give me my leave balance?

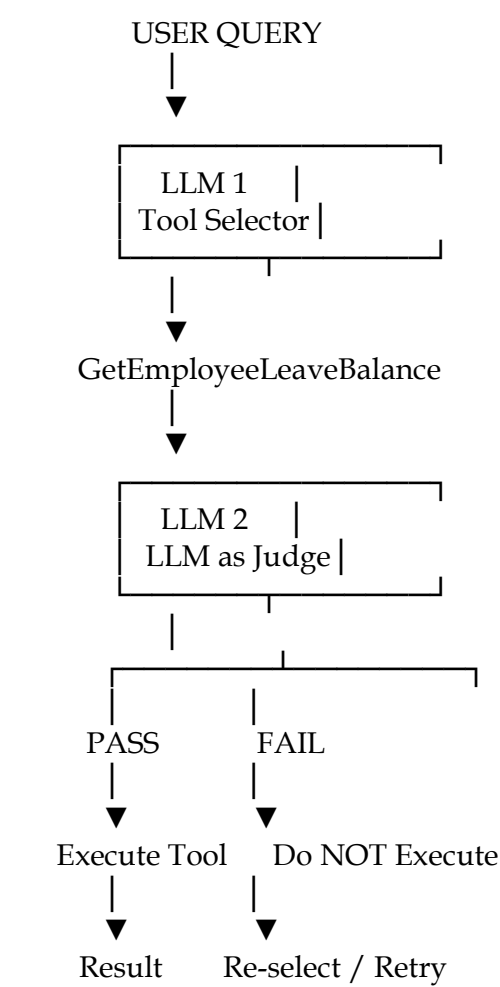
Selected Tool:
GetEmployeeLeaveBalance

Available Tools:
GetEmployeeLeaveBalance
GetSalary
GetTaxDetails
etc.

LLM 2 Response:

```
{
  "result": "PASS",
  "confidence": 0.95,
  "reason": "The user is asking for leave balance, and GetEmployeeLeaveBalance is
the correct tool."
}
```

Final Execution Flow



Why use LLM as Judge?

This approach provides an additional **validation layer before executing a tool**.

It can help us:

- Validate whether the correct tool was selected.
- Detect incorrect tool selection.
- Assign a **confidence score**.
- Prevent potentially incorrect tool execution.
- Log tool-selection accuracy.
- Compare different LLMs for tool-selection quality.

The architecture becomes:

LLM 1 = Decision Maker

LLM 2 = Judge / Validator

Application = Final Execution Controller